

LINUX SYSTEM PROGRAMMING · 综合编程练习

Linux 系统编程

综合编程题 10 道

覆盖知识点：Shell 编程 · 系统调用与文件描述符
进程管理（fork / wait / exit）· 特殊进程与 exec 函数族

题目来源：有道云笔记《linux系统编程》知识点整理与设计
2026 年 8 月

题目说明与知识点覆盖表

1. 共 10 道编程题，由易到难递进，第 10 题为综合大作业；建议独立完成后，再对照每题附带的参考代码框架检查。
2. 第 1~2 题使用 Shell 语言，第 3~10 题使用 C 语言 + Linux 系统调用；全部在 Ubuntu 终端环境下编译运行（gcc 编译、./a.out 执行）。
3. 建议用时 120 分钟，每题 10 分，总分 100 分。

题号	题目	核心知识点
1	Shell 日志分析脚本	变量 · 测试语句 · for · case
2	菜单式系统运维工具	while · case · read · 脚本结构
3	文件复制器 mycp	open/read/write/close · 描述符
4	printf 输出重定向	描述符 · dup2 · 缓冲区
5	fork 与父子关系	fork · getpid · 地址空间
6	exit 与 _exit 对比	exit · _exit · atexit
7	多进程非阻塞回收	waitpid · WNOHANG
8	僵尸与孤儿进程	僵尸 · 孤儿 · init 收养
9	exec 函数族应用	execl/execlp/execv
10	简易 Shell 综合实现	fork+exec+waitpid 综合

题目 1 Shell 日志分析脚本

考察知识点 | shell变量 · read · 测试语句 · for循环 · case分支 · 文件测试

【题目要求】

编写 Shell 脚本 log_analyzer.sh，实现一个日志分析工具：

1. 使用 read

从键盘读取一个日志目录路径，用测试语句判断该目录是否存在，不存在则输出错误信息并退出；

2. 使用 for 循环遍历目录下所有以 .log 结尾的文件，分别统计每个文件的总行数与包含 ERROR 关键字的行数；

3. 使用 case 分支将统计结果分级：0 条→正常，1~5 条→警告，超过 5 条→严重；

4. 输出汇总报告：文件名、总行数、ERROR 行数、故障级别；

5. 分别用 bash log_analyzer.sh 与 ./log_analyzer.sh 两种方式运行，并说明两种执行方式的区别。

【参考代码框架】

```
#!/bin/bash
read -p "请输入日志目录路径: " dir
[ -d "$dir" ] || { echo "目录不存在，退出"; exit 1; }

for f in "$dir"/*.log; do
    total=$(wc -l < "$f")
    err=$(grep -c "ERROR" "$f")
    case $err in
        0)    level="正常" ;;
        [1-5]) level="警告" ;;
        *)    level="严重" ;;
    esac
    echo "$f | 总行数:$total | ERROR:$err | 级别:$level"
done
```

【提示】

提示：测试语句有两种固定格式：test 操作符 文件名 与 [操作符 文件名] ([左右两侧必须各留一个空格)；case 分支的取值支持通配符*。

题目 2 菜单式系统运维工具

考察知识点 | while循环 · case分支 · read · 测试语句 · 函数与脚本结构

【题目要求】

编写 sys_menu.sh，实现一个循环菜单式运维小工具：

1. 使用 while 死循环显示功能菜单：1) 备份目录 2) 清理临时文件 3) 统计进程数 4) 退出；
2. 用 read 读取用户选择，用 case 分支分发到对应功能，非法输入给出提示后重新显示菜单；
3. 备份功能：接收一个目录路径，先测试其存在性，再用 cp -r 备份到 /backup 下并以日期命名；
4. 清理功能：使用 find 找出 /tmp 下 7 天前的 *.tmp 文件，交互式确认后删除；
5. 统计功能：使用 ps 统计当前系统进程总数并输出；
6. 输入 4 时 break 退出循环。

【参考代码框架】

```
#!/bin/bash
while true; do
    echo "===== 系统运维菜单 ====="
    echo "1) 备份目录  2) 清理临时文件"
    echo "3) 统计进程数 4) 退出"
    read -p "请选择操作: " choice
    case $choice in
        1) read -p "请输入要备份的目录: " dir
           [ -d "$dir" ] || { echo "目录不存在"; continue; }
           cp -r "$dir" "/backup/${basename "$dir"}-$(date +%Y%m%d)"
           echo "备份完成" ;;
        2) find /tmp -name "*.tmp" -mtime +7 -exec rm -i {} \; ;;
        3) echo "当前进程总数: $(ps -e | wc -l)" ;;
        4) echo "再见"; break ;;
        *) echo "无效选项，请重新输入" ;;
    esac
done
```

【提示】

提示：case 语句中 * 可匹配所有未列出的情况，用于处理非法输入；命令替换 \$(...) 可以把命令的输出作为字符串使用。

题目 3 系统调用实现文件复制器 mycp

考察知识点 | open · read · write · close · perror · O_CREAT/O_TRUNC · 文件描述符

【题目要求】

使用系统调用 open / read / write / close 实现命令 mycp，用法：./mycp 源文件 目标文件。

1. 以 O_RDONLY 打开源文件，以 O_WRONLY | O_CREAT | O_TRUNC 打开目标文件（权限 0644，并思考 umask 的影响）；
2. 循环调用 read 每次读取 1024 字节到缓冲区，再调用 write 写入目标文件，直到 read 返回 0（文件末尾）；
3. 所有系统调用失败时返回 -1，必须用 perror 输出具体错误原因后退出；
4. 复制完成后打印复制的总字节数，并正确 close 两个文件描述符；
5. 思考题：为什么 open 之后必须判断返回值？C 库函数 fopen 与系统调用 open 是什么关系（用户态/内核态）？

【参考代码框架】

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "用法: ./mycp 源文件 目标文件\n");
        return 1;
    }
    int fd_src = open(argv[1], O_RDONLY);
    if (fd_src == -1) { perror("open src"); return 1; }

    int fd_dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_dst == -1) { perror("open dst"); return 1; }

    char buf[1024];
    int n, total = 0;
    while ((n = read(fd_src, buf, sizeof(buf))) > 0) {
        if (write(fd_dst, buf, n) != n) { perror("write"); break; }
        total += n;
    }
    close(fd_src);
    close(fd_dst);
    printf("复制完成, 共 %d 字节\n", total);
    return 0;
}
```

【提示】

提示：每个程序启动时系统自动打开

0（标准输入）、1（标准输出）、2（标准错误）三个文件；内核为每个新打开的文件分配文件描述符表中最小可用的编号。

题目 4 printf 输出重定向到文件

考察知识点 | 文件描述符 · dup2 · close · printf 缓冲机制 · write

【题目要求】

完成笔记中的思考题：把 printf 的输出不再送到终端，而是写入指定的文件。

1. 用 open 创建/清空文件 out.txt (O_WRONLY | O_CREAT | O_TRUNC, 0644) ；
2. 调用 dup2(fd, 1) 将文件描述符 1（标准输出）重定向到 out.txt；
3. 连续调用 printf 输出若干行内容，观察内容是否进入了文件而不是终端；
4. 再调用 write(1, ...) 直接写入一行，验证系统调用不受缓冲约束；
5. 思考题：printf 与 write 的刷新机制有何不同（行缓冲/全缓冲）？为什么库函数之上还要设置缓冲区？

【参考代码框架】

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) { perror("open"); return 1; }

    dup2(fd, 1); /* 标准输出重定向到 out.txt */

    printf("这行 printf 内容会写入 out.txt 而不是终端\n");
    write(1, "write 直接写入，不受缓冲区约束\n", 22);

    close(fd);
    return 0;
}
```

【提示】

提示：dup2(oldfd, newfd) 的作用是把 newfd 变成 oldfd 的副本，此后两个描述符指向同一个文件，写 newfd 等同于写 oldfd。

题目 5 fork 进程创建与父子关系分析

考察知识点 | fork · getpid · getppid · sleep · 执行顺序不确定 · 地址空间

【题目要求】

编写程序，分析 fork 创建子进程后的父子关系：

1. main 中定义局部变量 int i = 100，然后调用 fork 创建子进程；
2. 父子进程分别输出自己的 PID、对方的 PID 以及变量 i 的值，观察 i 是否“共享”；
3. 让子进程 sleep(2) 后再退出，父进程 sleep(1) 后输出“父进程结束”，运行 3 次比较输出顺序；
4. 回答：fork 的返回值在父进程与子进程中分别是什么？为什么父子进程的执行顺序是不确定的？
- 5.

思考：子进程继承了父进程的哪些资源？子进程独有的又是什么？如何保证父进程一定比子进程晚退出？

【参考代码框架】

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    int i = 100;
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }

    if (pid == 0) { /* 子进程 */
        printf("子进程: 我的PID=%d 父PID=%d i=%d\n", getpid(), getppid(), i);
        sleep(2);
        printf("子进程即将退出\n");
    } else { /* 父进程 */
        printf("父进程: 我的PID=%d 子PID=%d i=%d\n", getpid(), pid, i);
        sleep(1);
        printf("父进程结束\n");
    }
    return 0;
}
```

【提示】

提示：fork 成功后子进程从 fork 的下一行代码开始复制执行；父子进程拥有独立的地址空间副本（写时复制），修改 i 互不影响。

题目 6 进程退出机制：exit 与 _exit 对比

考察知识点 | exit · _exit · atexit · 缓冲区刷新 · 函数指针

【题目要求】

编写两个程序，对比安全退出与立即退出的差异：

1. 程序 A：main 中先 printf("hello")（注意不要写换行符），再调用 atexit 注册清理函数 my_cleanup，最后调用 exit(3) 退出；
2. 程序 B：代码与 A 完全相同，仅把 exit(3) 改为 _exit(3)；
3. 分别运行两个程序，观察：① "hello" 是否被输出？② my_cleanup 是否被调用？③ 在 shell 中执行 echo \$? 得到的退出码是多少？
4. 解释差异产生的原因，并说明 exit 与 _exit 的本质区别（库函数 vs 系统调用、是否安全善后）。

【参考代码框架】

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void my_cleanup(void) {
    printf("atexit 注册的清理函数被自动调用\n");
}

int main() {
    printf("hello");    /* 注意：没有换行，行缓冲不会自动刷新 */
    atexit(my_cleanup); /* 注册退出清理函数 */
    exit(3);            /* 改成 _exit(3) 再对比观察 */
    /* _exit(3); */
}
```

【提示】

提示：printf 的输出在遇到换行、缓冲区满或进程安全退出时才会刷新；exit 会自动刷新缓冲区并调用 atexit 注册的回调，_exit 则直接陷入内核、不做任何善后。

题目 7 多子进程的创建与非阻塞回收

考察知识点 | fork · 循环创建 · waitpid · WNOHANG · 退出状态宏

【题目要求】

父进程一次创建 5 个子进程，并以非阻塞方式全部回收：

1. 在 for 循环中连续调用 fork 创建 5 个子进程（关键点：子进程创建后必须立即 break 或 _exit，防止 fork 爆炸）；
2. 每个子进程 sleep(i+1) 秒后，以退出码 10+i 调用 _exit 退出；
3. 父进程使用 waitpid(-1, &status, WNOHANG) 非阻塞轮询：有子进程结束则打印其 PID 与退出状态，没有则打印“暂无子进程退出”并 sleep(1)；
4. 统计并输出最终回收的子进程总数，确认 5 个子进程全部被回收；
5. 说明 wait 与 waitpid 的区别，以及子进程退出状态存放在 status 的哪几位、用什么宏取出。

【参考代码框架】

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    for (i = 0; i < 5; i++) {
        pid_t pid = fork();
        if (pid == 0) { /* 子进程：立即跳出循环，防止 fork 爆炸 */
            sleep(i + 1);
            _exit(10 + i);
        }
    }
    int cnt = 0;
    while (cnt < 5) {
        int status;
        pid_t r = waitpid(-1, &status, WNOHANG);
        if (r > 0) {
            printf("回收子进程 %d, 退出状态 %d\n", r, WEXITSTATUS(status));
            cnt++;
        } else {
            printf("暂无子进程退出，继续轮询...\n");
            sleep(1);
        }
    }
    printf("全部回收完成，共 %d 个子进程\n", cnt);
    return 0;
}
```

【提示】

提示：waitpid 的 pid 参数：>0 回收指定子进程，=0 回收同组子进程，=-1 回收任一子进程；退出状态存放在 status 的 8~15 位，用 WEXITSTATUS(status) 取出。

题目 8 僵尸进程与孤儿进程

考察知识点 | 僵尸进程 · 孤儿进程 · wait · init(1号进程) · 资源回收

【题目要求】

亲手制造并观察两类特殊进程：

1. 程序 Z（僵尸）：子进程 sleep(2) 后 _exit(0)，父进程 sleep(10) 期间不调用 wait。运行期间在另一个终端执行 ps -ef | grep a.out，观察是否存在状态为 Z (<defunct>) 的僵尸进程；10 秒后父进程调用 wait(NULL) 回收，再观察僵尸进程是否消失；
2. 程序 O（孤儿）：父进程直接 exit(0)，子进程 sleep(3) 后打印 getppid()，运行观察打印出的父进程号是否为 1（init 进程）；
3. 回答：为什么僵尸进程有危害、孤儿进程没有危害？内核是如何处理孤儿进程的？

【参考代码框架】

```
/* 程序Z：制造并回收僵尸进程 */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) { sleep(2); _exit(0); } /* 子进程先退出 */
    sleep(10);                          /* 期间用 ps -ef 观察僵尸 */
    wait(NULL);                          /* 回收后僵尸消失 */
    printf("已回收子进程\n");
    return 0;
}

/* 程序O：制造孤儿进程 */
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        sleep(3);
        printf("我是孤儿，现在的父进程是 %d\n", getppid());
    } else {
        printf("父进程先退出\n");
        exit(0);
    }
    return 0;
}
```

【提示】

提示：子进程终止后，其 PCB 仍残留在内核中等待父进程回收，此时进程处于僵尸态；孤儿进程会被内核过继给 1 号进程（init），由 init 统一回收，因此没有危害。

题目 9 exec 函数族的应用

考察知识点 | exec家族 · execl / execlp / execv · fork · wait · 路径查找

【题目要求】

在一个进程中启动另一个进程，体验 exec 函数族：

1. 父进程 fork 创建子进程；
2. 在子进程中分别尝试三种方式启动 ls 命令：① execl("/bin/ls", "ls", "-l", NULL)；② execlp("ls", "ls", "-l", NULL)；③ 构造 char *argv[] = {"ls", "-l", NULL} 后用 execv("/bin/ls", argv)；
3. 在 exec 调用前后各加一行 printf，观察 exec 之后的代码是否执行，并说明原因；
4. 父进程用 wait 回收子进程，并打印子进程的退出状态；
5. 说明后缀 l / v / p / e 各自的含义，以及为什么 7 个函数中只有 execve 是真正的系统调用。

【参考代码框架】

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) { /* 子进程 */
        printf("子进程：即将 exec\n");
        execlp("ls", "ls", "-l", NULL); /* 换成 execl / execv 再对比 */
        perror("exec"); /* 只有 exec 失败才会执行到这里 */
        _exit(1);
    }
    int status;
    wait(&status); /* 父进程回收 */
    printf("子进程退出状态: %d\n", WEXITSTATUS(status));
    return 0;
}
```

【提示】

提示：exec 成功后调用进程的代码会被新程序完全覆盖，exec 之后的代码永远不会执行；只有 exec 失败（返回 -1）时才会继续执行后续代码。

题目 10 综合大作业：实现简易 Shell (myshell)

考察知识点 | 综合应用 · 字符串解析 · fork · execvp · waitpid · 内建命令 · 循环

【题目要求】

综合运用本课程全部知识点，实现一个迷你命令行解释器 myshell：

1. 死循环中打印提示符 mysh\$, 用 fgets 读取用户输入，并去掉末尾换行符；
2. 输入 exit 时退出程序，空行则继续循环；
3. 使用 strtok 按空格拆分命令与参数（最多支持 8 个参数，参数数组以 NULL 结尾）；
4. fork 创建子进程，子进程调用 execvp(args[0], args) 执行命令；
5. 父进程 waitpid 阻塞回收子进程，打印退出状态后回到提示符；
6. 加分项：实现内建命令 cd（调用 chdir 改变当前工作目录）；
7. 扩展思考：你的 myshell 相比系统 bash 还缺少哪些功能？“后台执行 &”应该如何实现？

【参考代码框架】

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_CMD 128
#define MAX_ARG 8

int main() {
    char cmd[MAX_CMD];
    while (1) {
        printf("mysh$ ");
        fflush(stdout);
        if (fgets(cmd, MAX_CMD, stdin) == NULL) break; /* Ctrl+D 退出 */
        cmd[strcspn(cmd, "\n")] = '\0'; /* 去掉换行 */
        if (strcmp(cmd, "exit") == 0) break; /* 内建命令 exit */
        if (cmd[0] == '\0') continue; /* 空行继续 */

        char *args[MAX_ARG + 1] = {0};
        int n = 0;
        char *tok = strtok(cmd, " ");
        while (tok && n < MAX_ARG) { args[n++] = tok; tok = strtok(NULL, " "); }
        args[n] = NULL;

        if (strcmp(args[0], "cd") == 0) { /* 内建命令 cd */
            if (chdir(args[1]) != 0) perror("cd");
            continue;
        }

        pid_t pid = fork();
        if (pid == 0) {
            execvp(args[0], args); /* 子进程执行命令 */
            perror("execvp");
            _exit(127);
        }
        int status;
        waitpid(pid, &status, 0); /* 父进程回收 */
        printf("[退出状态 %d]\n", WEXITSTATUS(status));
    }
    printf("bye~\n");
    return 0;
}

```

【提示】

提示：这是一个完整的“读取—解析—创建—执行—回收”循环，恰好串起 shell 解释器、fork、exec、waitpid 四大知识模块，是本次练习的压轴题。

评分参考

程序可正确编译并运行（无语法错误、无段错误）	4 分
功能完整，覆盖题目要求的全部功能点	3 分
边界处理与错误处理（参数校验、返回值为 -1 的判断、perror 提示）	2 分
代码规范、有注释、命名清晰	1 分

注：第 10 题综合大作业权重更高，可单独按“命令解析 3 分 + 进程创建执行 4 分 + 回收与健壮性 3 分”评分。

完成顺序建议：1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10，前 9 题是第 10 题的必要铺垫。